

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчёт по лабораторной работе №5
«Сортировки»

Выполнила работу:
Абаянцева Евгения
Академическая группа №J3110
Принято:
...

Санкт-Петербург
2024 г.

Введение

Цель – написать и проанализировать сортировки.

Задачи:

1. Odd-Even Sort;
2. Merge Sort;
3. Counting Sort.

Теоретическая подготовка

Данная задача предполагает написание кода для трёх сортировок.

Использованные типы данных для этой задачи – целые числа (тип `int`), строки (тип `string`), массивы (тип `vector`).

Реализация

Для реализации сортировок требуются следующие библиотеки:

1. Библиотека `iostream` – эта основная библиотека C++ для ввода и вывода текста с консоли;
2. Библиотека `cstdlib` – эта библиотека включает в себя пространство имён в `std`;
3. Библиотека `time.h` – эта библиотека определяет работу со временем и датой;
4. Библиотека `vector` – эта библиотека определяет вектор шаблона класса контейнера и несколько вспомогательных шаблонов;
5. Библиотека `string` – эта библиотека определяет шаблон `basic_string` класса контейнера и различные вспомогательные шаблоны.;
6. Библиотека `algorithm` – эта библиотека определяет функции шаблона контейнера стандартной библиотеки C++, которые выполняют алгоритмы.

Реализация была проведена для трёх сортировок: Odd-Even Sort имеет алгоритм через сравнение элементов на чётных и нечётных местах массива, Merge Sort предполагает алгоритм через разбиение массива на мелкие и сравнение их, Counting Sort имеет алгоритм только при

натуральных числах при использовании массива, в котором элементов столько же, сколько составляет максимальный элемент в изначальном массиве.

Особенность реализации: три сортировки с разными алгоритмами.

Экспериментальная часть

1. Память сортировки Odd-Even: $O(n^2)$.
Память сортировки Merge Sort: $O(n)$.
Память сортировки Counting Sort: $O(k)$.
2. Сложность сортировки Odd-Even Sort: $O(n) = n^2$.
Сложность сортировки Merge Sort: $O(n) = n \log n$.
Сложность сортировки Counting Sort: $O(n) = n + k$.

Лучшие случаи для каждой сортировки:

1. Odd-Even Sort: когда при первой пробежке весь массив отсортируется. Сложность – $O(n)$.
2. Merge Sort: когда все элементы списка уже отсортированы. Сложность – $O(n \cdot \log n)$.
3. Counting Sort: когда все элементы массива равны. Сложность – $O(n)$.

Худшие и средние случаи для каждой сортировки:

1. Odd-Even Sort: когда одной пробежки по массиву недостаточно. Сложность – $O(n^2)$.
2. Merge Sort: когда элементы не отсортированы по массиву. Сложность – $O(n \cdot \log n)$.
3. Counting Sort: когда элементы различны и большие расходы в значениях (тогда выделяется больший массив). Сложность – $O(n + k)$.

Таблица для генерации графика с массивом от 1000 до 1e6 с шагом в 1000:

Таблица 1 – Результаты для построения графика

Размер массива	Odd-Even Sort	Merge Sort	Counting Sort
1000	0,258	0,108	0,1075
2000	1,006	0,154	0,1391
3000	2,281	0,162	0,1691
4000	4,675	0,24	0,1391
5000	8,849	0,369	0,1627
...	...	...	...
500000	310,25	4,271	0,9856
...	...	...	...
750000	410,5684	5,4964	1,2684
...	...	...	...
999000	547,605	6,5243	1,5056
1000000	560,725	6,5572	1,5132

Заключение

При полном неотсортированном массивом натуральных чисел лучше использовать Counting Sort. Если всё уже более-менее рассортировано, то лучше использовать Odd-Even Sort или Merge Sort.

Сортировку Counting Sort лучше использовать остальных, когда мы работаем с массивом натуральных чисел. Odd-Even Sort – если у нас малый массив с не натуральными числами, Merge Sort – если больший.

Приложения

```

1 #include <iostream>
2 #include <cstdlib>
3 #include <time.h>
4 #include <vector>
5 #include <string>
6 #include <algorithm>
7
8 #define N 10000 // максимальный размер вектора
9 #define ABSLIMIT 100000 // предел значений элементов массива
10
11 #define SORT__ODD_EVEN 1
12 #define SORT__MERGING 2
13 #define SORT__COUNTING 3
14
15 std::vector<std::string> sort_names = { "", "Odd_Even_Sort", "Merge_Sort", "
    Counting_Sort" };
16
17 void print_vector(const std::vector<int>& mas, const std::string msg) {
18     int n = 0;
19
20     std::cout << msg << "\nSize: " << mas.size() << "\n";

```

```

21
22     for (int num : mas) {
23         std::cout << num << " ";
24         n++;
25         if (n > 20) {
26             std::cout << "...";
27             break;
28         }
29     }
30 }
31
32
33 std::vector<int> Odd_Even_Sort( std::vector<int>& mas, int& coun ) {
34     int n = mas.size();
35     int isSorted = 0;
36     coun += sizeof(n) + sizeof(isSorted);
37
38     while (isSorted == 0) {
39         isSorted = 1;
40         for (int i = 1; i < n - 1; i += 2) {
41             if (mas[i] > mas[i + 1]) {
42                 std::swap(mas[i], mas[i + 1]);
43                 isSorted = 0;
44             }
45         }
46         for (int i = 0; i < n - 1; i += 2) {
47             if (mas[i] > mas[i + 1]) {
48                 std::swap(mas[i], mas[i + 1]);
49                 isSorted = 0;
50             }
51         }
52     }
53
54     // coun = sizeof(n) + sizeof(isSorted);
55     // std::cout << coun << "\n";
56
57     return mas;
58 }
59
60
61 std::vector<int> Merge_Sort(std::vector<int>& mas, int& coun) {
62     if (mas.size() <= 1) {
63         return mas;
64     }
65
66     std::vector<int> lmas(mas.begin(), mas.begin() + mas.size() / 2);
67     std::vector<int> rmas(mas.begin() + mas.size() / 2, mas.end());
68     coun += sizeof(lmas) + sizeof(rmas);
69
70     lmas = Merge_Sort(lmas, coun);
71     rmas = Merge_Sort(rmas, coun);
72
73     int n = 0, m = 0, k = 0;
74     coun += sizeof(n) + sizeof(m) + sizeof(k);
75     std::vector<int> pob_mas(mas.size());
76     coun += sizeof(pob_mas);
77
78     while (n < lmas.size() && m < rmas.size()) {
79         if (lmas[n] <= rmas[m]) {
80             pob_mas[k] = lmas[n];

```

```

81     n++;
82 }
83 else {
84     pob_mas[k] = rmas[m];
85     m++;
86 }
87 k++;
88 }
89
90 while (n < lmas.size()) {
91     pob_mas[k] = lmas[n];
92     n++;
93     k++;
94 }
95
96 while (m < rmas.size()) {
97     pob_mas[k] = rmas[m];
98     m++;
99     k++;
100 }
101
102 for (size_t i = 0; i < mas.size(); i++) {
103     mas[i] = pob_mas[i];
104 }
105
106 return mas;
107 }
108
109
110 std::vector<int> Counting_Sort( const std::vector<int>& mas3, int& coun ) {
111     int maxElement = *std::max_element(mas3.begin(), mas3.end());
112     int countArrayLength = maxElement + 1;
113     coun += sizeof(maxElement) + sizeof(countArrayLength);
114
115     std::vector<int> countArray(countArrayLength, 0);
116     coun += sizeof(countArray);
117
118     for (int el : mas3) {
119         countArray[el]++;
120     }
121
122     for (int i = 1; i < countArrayLength; i++) {
123         countArray[i] += countArray[i - 1];
124     }
125
126     std::vector<int> outputArray(mas3.size());
127     int i = mas3.size() - 1;
128     coun += sizeof(outputArray) + sizeof(i);
129     while (i >= 0) {
130         int currentEl = mas3[i];
131         countArray[currentEl]--;
132         int newPosition = countArray[currentEl];
133         outputArray[newPosition] = currentEl;
134         i--;
135     }
136
137     return outputArray;
138 }
139
140

```

```

141 // функция генерации случайного числа из диапазона от range_min до range_max
    включительно
142 int rrand(int range_min, int range_max) {
143     return rand() % (range_max - range_min + 1) + range_min;
144 }
145
146
147 std::vector<int> Create_Massiv(int size, int limit_1, int limit_2) {
148     std::vector<int> mas;
149     int min_value, max_value;
150
151     srand(static_cast<unsigned int>(time(NULL)));
152
153     if (limit_1 < limit_2) {
154         min_value = limit_1;
155         max_value = limit_2;
156     }
157     else {
158         min_value = limit_2;
159         max_value = limit_1;
160     }
161     // инициализация массива случайными значениями из диапазона -ABSLIMIT..ABSLIMIT
162     for (unsigned int i = 0; i < size; i++)
163         mas.push_back(rrand(min_value, max_value));
164
165     return mas;
166 }
167
168
169 std::vector<int> Test_Sort(int size, int sort_code) {
170     std::vector<int> res = {0, 0, 0, 0, 0, 0}; // 0 - размер памяти, времена: 1 -
        начало теста, 2 - создание, 3 - печать, 4 - сортировка, 5 - печать завершения
171     std::vector<int> sorted_mas;
172     int mem_sz;
173
174     res[0] = sizeof(res);
175     res[1] = clock();
176     std::vector<int> mas = Create_Massiv(size, rrand(0, ABSLIMIT), rrand(0,
        ABSLIMIT));
177     res[2] = clock();
178     print_vector(mas, "\n" + sort_names[sort_code] + ":");
179     res[3] = clock();
180     switch(sort_code) {
181         case SORT__ODD_EVEN:
182             sorted_mas = Odd_Even_Sort(mas, mem_sz);
183             break;
184         case SORT__MERGING:
185             sorted_mas = Merge_Sort(mas, mem_sz);
186             break;
187         case SORT__COUNTING:
188             sorted_mas = Counting_Sort(mas, mem_sz);
189             break;
190         default:
191             break;
192     }
193     res[0] += mem_sz;
194     res[4] = clock();
195     print_vector(sorted_mas, "");
196     res[5] = clock();
197     std::cout << (res[4] - res[3]) << "\n";

```

```

198
199     return res;
200 }
201
202
203 void update_stat(std::vector<int>& stat, std::vector<int>& stat1, int sort_index
    ) {
204     stat[0] += stat1[0];
205     stat[1] += (stat1[5] - stat1[1]);
206     stat[2] += (stat1[2] - stat1[1]);
207     stat[3] += (stat1[3] - stat1[2] + stat1[5] - stat1[4]);
208     stat[4] += (stat1[4] - stat1[3]);
209     stat[sort_index] += (stat1[4] - stat1[3]);
210 }
211
212
213 int main() {
214     // int coun = 0;
215     std::vector<int> stat = { 0, 0, 0, 0, 0, 0, 0, 0, 0 }; // 0 - размер памяти,
        времена: 1 - общее время, 2 - создание, 3 - печать, 4 - сортировка, 5 - ODD_EVEN,
        6 - MERGE, 7 - COUNTING
216     std::vector<int> stat_test; // 0 - размер памяти, времена: 1 - начало теста, 2 -
        создание, 3 - печать1, 4 - сортировка, 5 - печатьзавершение2/
217     std::vector<int> stat_1;
218     std::vector<int> stat_2;
219     std::vector<int> stat_3;
220
221     for (int size = 1000; size <= 10000; size += 1000) {
222         stat_test = Test_Sort(size, SORT__ODD_EVEN);
223         update_stat(stat, stat_test, 5);
224         stat_1.push_back(stat_test[4] - stat_test[3]);
225
226         stat_test = Test_Sort(size, SORT__MERGING);
227         update_stat(stat, stat_test, 6);
228         stat_2.push_back(stat_test[4] - stat_test[3]);
229
230         stat_test = Test_Sort(size, SORT__COUNTING);
231         update_stat(stat, stat_test, 7);
232         stat_3.push_back(stat_test[4] - stat_test[3]);
233     }
234
235     std::cout << "\nUsing memory: " << (stat[0] + sizeof(stat) + sizeof(stat_test)
        + sizeof(stat_1) + sizeof(stat_2) + sizeof(stat_3)) << " bt";
236     for (int i = 0; i < 10; i++) {
237         std::cout << "\n" << (i + 1) * 1000
238             << "\t" << stat_1[i] << "\t" << stat_2[i] << "\t" << stat_3[i]
239             << "\t" << ((float)stat_1[i]) / CLOCKS_PER_SEC << "\t" << ((float)stat_2
                [i]) / CLOCKS_PER_SEC << "\t" << ((float)stat_3[i]) / CLOCKS_PER_SEC;
240     }
241     return 0;
242 }

```

Листинг 1 – код файла lab-5.cpp

График работы всех сортировок:

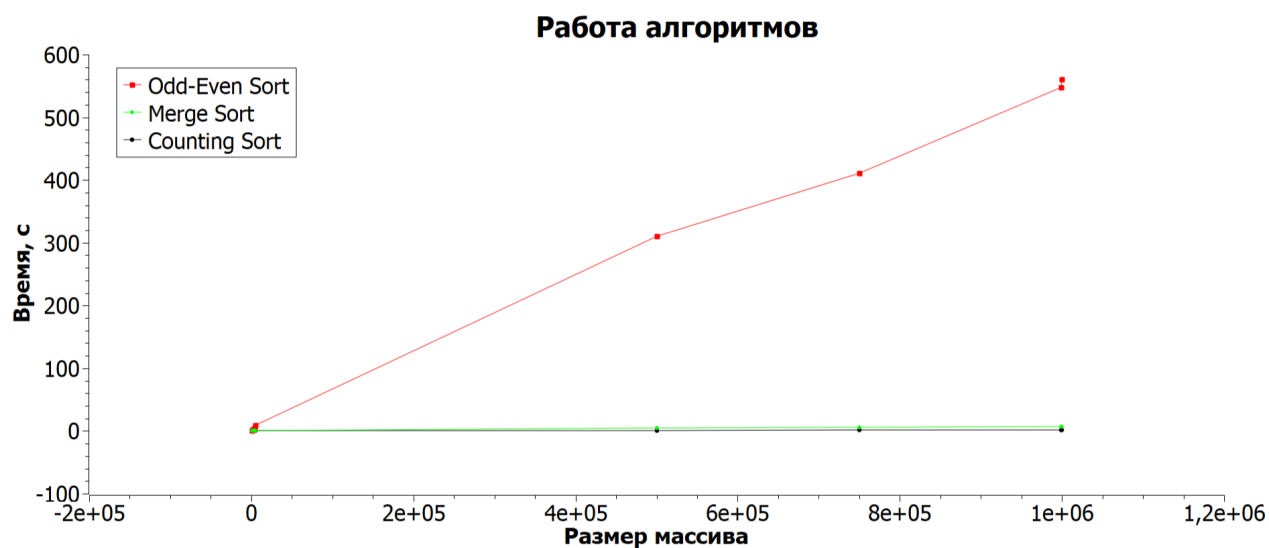


Рис. 1 – график работы всех сортировок

Box Plot для $1e4$:

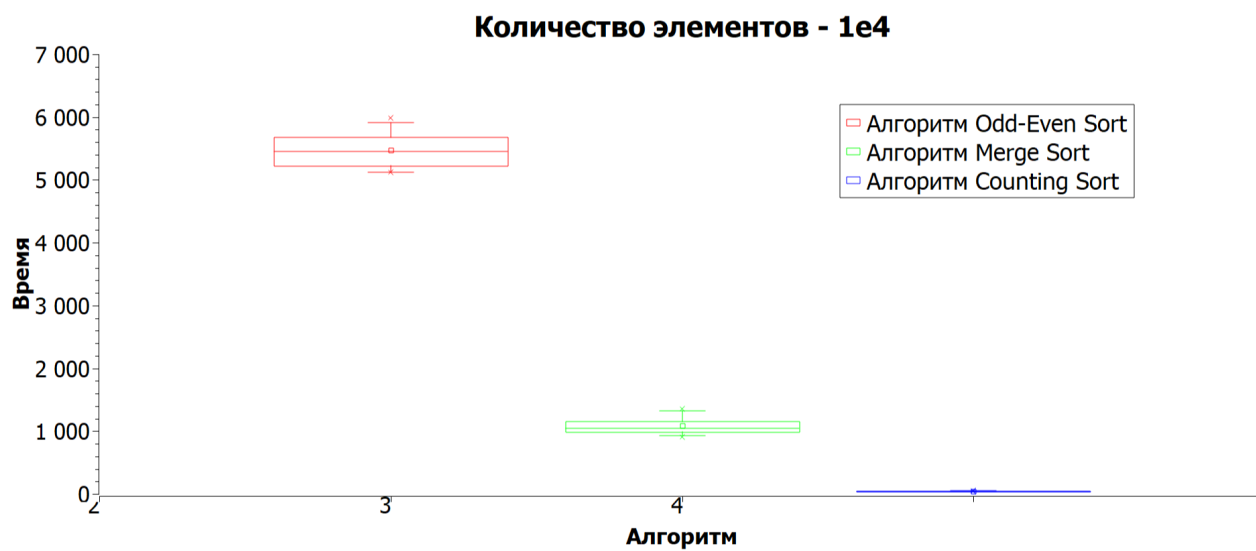


Рис. 2 – box plot для $1e4$

Box Plot для $1e5$:

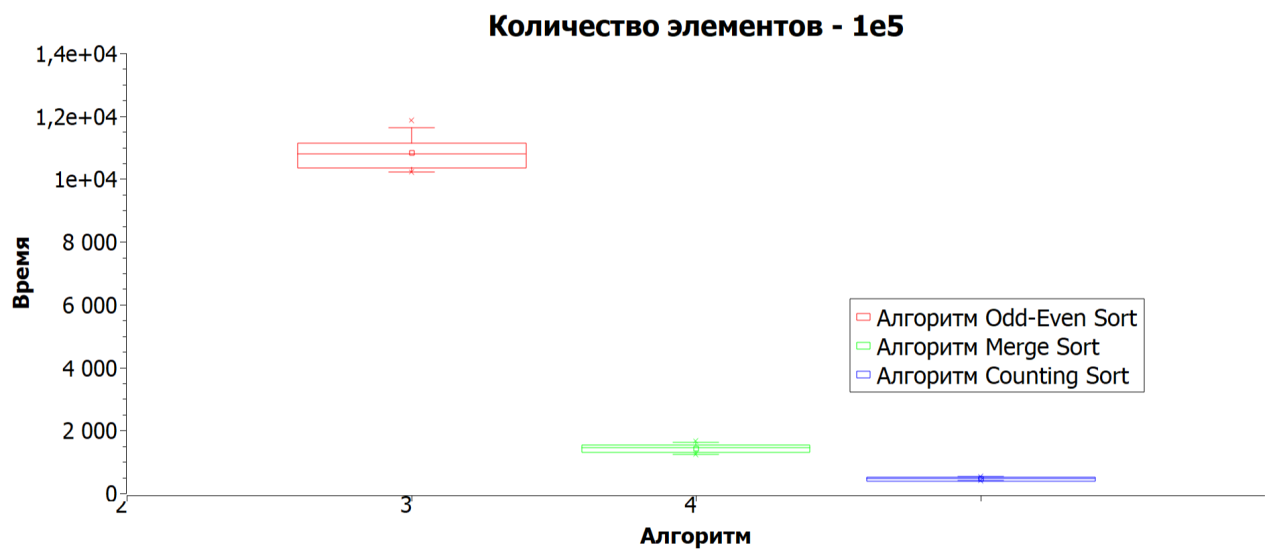


Рис. 3 – box plot для 1e5